
ProcessLens

An AI-Assisted Platform for Discovering
How Business Processes Really Work

Project Proposal

Data-Driven Process Discovery and Optimization

PREPARED BY

Moaz Hassan

September 4, 2026

Contents

Project Summary	2
1 Problem Statement	2
2 Who Faces This Problem	2
3 Proposed Solution	2
4 How the System Works	3
4.1 Stage 1: Collect the Event Log	3
4.2 Stage 2: Validate and Clean the Data	3
4.3 Stage 3: Discover the Process	3
4.4 Stage 4: Measure Performance	3
4.5 Stage 5: Predict Delays	4
4.6 Stage 6: Explain and Recommend	4
5 Illustrative Example	4
6 Why This Project Is Different	4
7 Core Features	5
8 System Architecture	5
9 Technology Stack	6
10 Team Roles	6
11 First Working Version	7
12 Evaluating the System	7
13 Possible Future Additions	8
14 Possible Challenges	8
15 Why This Project Should Be Selected	8

Project Summary

ProcessLens is a platform that studies the recorded history of a business process, such as the timestamps of every step a request, ticket, or order went through, and reconstructs how that process actually happens in practice. It highlights the steps that cause delays, the steps that get repeated unnecessarily, and the paths that stray from the intended procedure. It then predicts which new cases are likely to run late and produces plain-language, evidence-based recommendations for improvement. Most of the analysis is handled by established process-mining and machine-learning methods; a language model is used only to explain the results in clear language, keeping the system fast, affordable, and easy to justify.

1 Problem Statement

Almost every organization has a written procedure for how work should move through it: a request is submitted, reviewed, approved, and completed. In practice, the real sequence of events is rarely this clean. Steps get skipped, extra approvals are added, documents are returned for correction, and some cases wait far longer than others for no clear reason.

Over time, the process that is actually followed drifts away from the process that was originally documented, and because nobody is systematically tracking every case, this drift goes unnoticed. The organization is left knowing that something is slow, without knowing exactly where the delay happens, how often it happens, or why.

2 Who Faces This Problem

- Organizations that run multi-step approval or request workflows, such as purchase requests, leave approvals, or refunds.
- Operations and quality teams that are responsible for finding and removing inefficiencies.
- IT support and helpdesk teams whose ticket-handling process no longer matches their documented service standard.
- Administrative departments that process large volumes of similar, repeated cases, such as admissions or invoice processing.

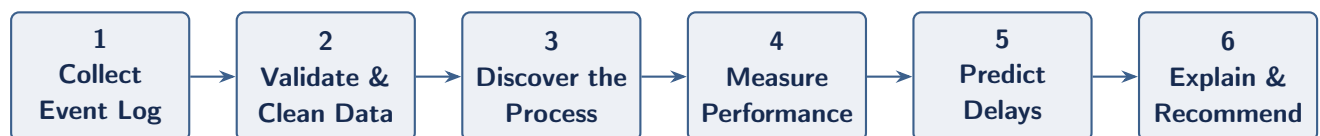
3 Proposed Solution

ProcessLens takes historical event data, meaning a simple record of which activity happened, for which case, and at what time, and produces four outcomes:

1. A visual map of the process as it is actually carried out, including the common variations that occur alongside the main path.
2. A ranked list of the slowest and most repeated steps, along with the amount of time and the number of cases they affect.
3. A prediction of which currently open cases are at risk of finishing late.
4. A plain-language summary of these findings, together with practical, evidence-based suggestions for improvement.

4 How the System Works

The system processes data in six stages, moving from raw event records to a clear, explained recommendation.



4.1 Stage 1: Collect the Event Log

The system accepts a simple table of events. Each row records a case identifier, the activity that took place, a timestamp, and, when available, the person or department responsible. For the first version, data is provided as a CSV or Excel file.

4.2 Stage 2: Validate and Clean the Data

Before any analysis begins, the system checks the data for common problems: missing case identifiers, missing or incorrectly ordered timestamps, duplicate entries, and activities that appear in an impossible sequence. Any issues found are reported to the user before the process is analyzed.

4.3 Stage 3: Discover the Process

The system examines the events belonging to every case and identifies the sequence of steps that most cases follow, along with the alternative sequences, or *variants*, that a smaller number of cases follow instead. This produces a process map that reflects what genuinely happened, not what was written in a manual.

4.4 Stage 4: Measure Performance

For every step in the process, the system calculates the time spent actively working on it and the time spent waiting before it began. It ranks the steps by how much they contribute to the

total delay, identifying clear bottlenecks, and it separately detects steps that occur more than once for the same case, which usually signals rework caused by an earlier problem.

4.5 Stage 5: Predict Delays

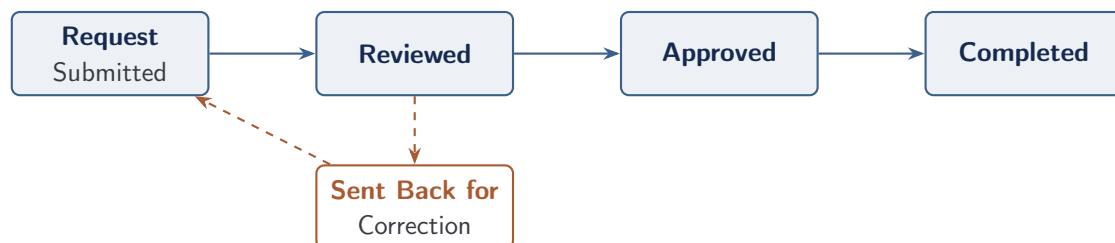
Using the patterns found in past cases, a machine-learning model estimates the likelihood that a currently open case will finish later than expected, and flags unusual cases that do not resemble the typical pattern.

4.6 Stage 6: Explain and Recommend

A language model is used at this final stage only. It receives the structured results calculated in the previous stages, such as which step causes the most delay and how often rework occurs, and turns them into a short, plain-language explanation together with a specific, measurable recommendation for improvement.

5 Illustrative Example

The figure below illustrates the kind of result ProcessLens produces for a typical request-handling process. Most cases (72%) follow the expected path. A smaller group of cases (18%) are sent back for an additional review, and this extra loop is the main source of delay.



Solid arrows: main path (72% of cases). Dashed arrows: rework loop (18% of cases).

A supporting performance table, generated automatically from the same data, would look like this:

Step	Average Wait	Contribution to Delay
Review	4.2 hours	Medium
Approval	31.7 hours	High
Processing	0.7 hours	Low

6 Why This Project Is Different

Many student AI projects rely on a language model to interpret raw data directly, which makes results harder to verify and expensive to run at scale. ProcessLens keeps the language model out

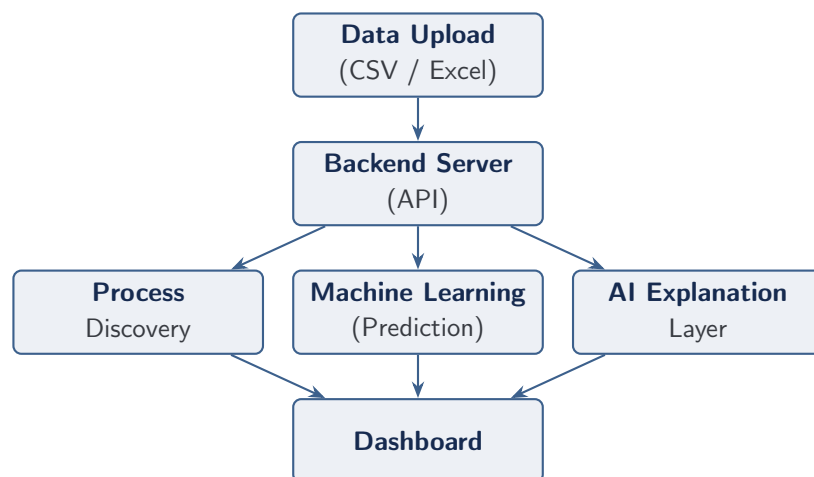
of the core analysis. Reconstructing the process, measuring delays, and detecting repeated steps are done with established, deterministic process-mining and statistics methods. The language model is used only once, at the final stage, to explain results that have already been calculated with full accuracy. This keeps the system fast and inexpensive to run, and every explanation it produces can be traced back to a specific, verifiable number.

7 Core Features

- Automatic discovery of the process as it is actually carried out, including its common variants.
- An interactive process map showing the sequence of steps and how frequently each path is followed.
- Bottleneck detection, ranking steps by their average waiting time and their share of total delay.
- Rework detection, identifying steps that are repeated for the same case and estimating how often this happens.
- Delay prediction for open cases, based on patterns learned from completed cases.
- Anomaly detection, flagging cases whose behavior does not match the typical pattern.
- A plain-language summary of findings, generated from the calculated results rather than from raw data.
- Evidence-based recommendations for improving the process, each tied to a specific measurement.

8 System Architecture

The system is organized into layers, from data intake to the final dashboard presented to the user.



1. **Data Upload:** The user provides an event log as a CSV or Excel file.
2. **Backend Server:** Coordinates data validation, storage, and the three analysis components below.
3. **Process Discovery:** Reconstructs the real process map and its variants from the event log.
4. **Machine Learning:** Predicts delay risk for open cases and flags unusual ones.
5. **AI Explanation Layer:** Converts the structured results into a plain-language summary and recommendations.
6. **Dashboard:** Displays the process map, performance metrics, predictions, and recommendations together.

9 Technology Stack

Component	Technology
Frontend	Next.js with Tailwind CSS, using a graph library to render the process map
Backend	Python with FastAPI
Data Processing	Pandas
Process Discovery	An established open-source process-mining library (such as PM4Py)
Machine Learning	scikit-learn (delay prediction and anomaly detection)
Database	PostgreSQL, hosted on Supabase
AI Explanation Layer	A free-tier or locally hosted language model

All core components are open-source or available on free tiers, so the first working version can be built without a paid infrastructure budget.

10 Team Roles

The project is structured so that it can be completed by a single person if necessary, since the process-mining library handles most of the core technical work. Where a team is available, responsibilities can be divided as follows.

Role	Responsibility
Data & Process Mining	Prepares and cleans event-log data and configures process discovery and variant analysis.
Machine Learning	Builds and evaluates the delay-prediction and anomaly-detection models.
Backend & AI	Builds the API, manages the database, and implements the AI explanation layer.
Frontend	Builds the dashboard, including the interactive process map and performance views.

11 First Working Version

The first working version will:

- Accept an event log in CSV or Excel format.
- Automatically generate a process map, including its main variants.
- Calculate waiting time and processing time for each step, and rank the resulting bottlenecks.
- Detect steps that are repeated for the same case.
- Predict which open cases are likely to be delayed.
- Produce a plain-language summary of the findings with at least one concrete recommendation.

For demonstration purposes, a realistic event log will be generated by the project team, modeled on a typical request-and-approval process, so that no external or sensitive data is required.

12 Evaluating the System

To confirm that ProcessLens works as intended, its output will be checked against data where the correct answer is already known.

- **Process discovery:** Compare the process map produced by the system against the process used to generate the sample data.
- **Bottleneck detection:** Introduce a deliberate delay into a specific step of the sample data and confirm the system identifies it.
- **Anomaly and delay prediction:** Measure standard classification metrics, such as precision, recall, and accuracy, on cases with a known outcome.
- **Recommendation quality:** Compare the recommendations generated by the system against the improvement that was expected for each planted problem.

13 Possible Future Additions

- Direct connection to an organization's existing systems, so event data is collected automatically instead of being uploaded manually.
- Comparison between the documented process and the observed process, showing exactly where they differ.
- Simulation of a proposed process change, to estimate its likely effect before it is put into practice.
- Tracking how a process changes over time, rather than analyzing a single snapshot.
- Suggesting which repetitive manual steps could be automated, and generating a corresponding workflow specification.

14 Possible Challenges

- **Presenting a process map clearly to a non-technical audience.** This will be addressed by keeping the dashboard visual and simple, and letting the plain-language summary carry the main explanation.
- **Obtaining realistic activity data.** This will be addressed by generating a believable sample process for the demonstration, rather than depending on real organizational data.
- **Keeping the AI explanation grounded in the actual numbers.** This will be addressed by giving the language model only the already-calculated results, rather than the raw event log, so it cannot introduce unsupported claims.

15 Why This Project Should Be Selected

- It addresses a real and widespread problem: the process an organization actually follows is rarely identical to the one it has documented.
- It is efficient to run, since the core analysis does not depend on a language model.
- It combines several genuine technical areas, including data processing, process mining, machine learning, and applied generative AI, giving the project real technical depth.
- It is realistic to complete within the available timeline, and can be built even by a single team member if needed.
- The demonstration is clear and visual: showing an organization's real, messy process automatically emerge from simple activity data is easy to present and easy to understand.